There's also the bang variant `except!` that removes keys in the very receiver.
Defined in `active_support/core_ext/hash/except.rb`.

11.4.2 `transform_keys` and `transform_keys!`

The method `transform_keys` accepts a block and returns a hash that has applied the block operations to each of the keys in the receiver:

```
{nil => nil, 1 => 1, a: :a}.transform_keys { |key| key.to_s.upcase }
# => {"" => nil, "A" => :a, "1" => 1}
```

In case of key collision, one of the values will be chosen. The chosen value may not always be the same given the same hash:

```
{"a" => 1, a: 2}.transform_keys { |key| key.to_s.upcase }
# The result could either be
# => {"A"=>2}
# or
# => {"A"=>1}
```

This method may be useful for example to build specialized conversions. For instance `stringify_keys` and `symbolize_keys` use `transform_keys` to perform their key conversions:

```
def stringify_keys
  transform_keys { |key| key.to_s }
end
...
def symbolize_keys
  transform_keys { |key| key.to_sym rescue key }
end
```

There's also the bang variant `transform_keys!` that applies the block operations to keys in the very receiver.

Besides that, one can use `deep_transform_keys` and `deep_transform_keys!` to perform the block operation on all the keys in the given hash and all the hashes nested into it. An example of the result is:

```
{nil => nil, 1 => 1, nested: {a: 3, 5 => 5}}.deep_transform_keys { |key|
key.to_s.upcase }
# => {""=>nil, "1"=>1, "NESTED"=>{"A"=>3, "5"=>5}}
```

Defined in `active_support/core_ext/hash/keys.rb`.

11.4.3 `stringify_keys` and `stringify_keys!`

The method `stringify_keys` returns a hash that has a stringified version of the keys in the receiver. It does so by sending `to_s` to them:

```
{nil => nil, 1 => 1, a: :a}.stringify_keys
# => {"" => nil, "a" => :a, "1" => 1}
```

In case of key collision, one of the values will be chosen. The chosen value may not always be the same given the same hash:

```
{"a" => 1, a: 2}.stringify_keys
# The result could either be
# => {"a"=>2}
# or
# => {"a"=>1}
```

This method may be useful for example to easily accept both symbols and strings as options. For instance `ActionView::Helpers::FormHelper` defines:

```
def to_check_box_tag(options = {}, checked_value = "1", unchecked_value
= "0")
  options = options.stringify_keys
  options["type"] = "checkbox"
  ...
end
```

The second line can safely access the "type" key, and let the user to pass either `:type` or "type".

There's also the bang variant `stringify_keys!` that stringifies keys in the very receiver.